probability.thresholds

Bao Han Ngo

2026-05-11

```
library(readxl)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v readr     2.1.5
## v ggplot2   3.5.2     v stringr   1.5.1
## v lubridate 1.9.4     v tibble    3.3.0
## v purrr     1.1.0     v tidyr     1.3.1

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
fetal.dat <- read_excel("../Data/raw.fetal.data.xlsx")
fetal.dat <- fetal.dat[-1,] #empty first row
#each row is its own id
#variables we'll actually use (including NSP)
#removing date, id, filename, FEB, b, e, AC, DR, SUSP, class, and other classification variables not in
fetal <- fetal.dat |>
  dplyr::select(-c("FileName","Date","SegFile","b", "e", "LBE", "AC", "DR", "A",
                   "B", "C", "D", "E", "AD","DE","LD","FS","SUSP","CLASS"))
#make NSP a factor (only categorical variable in our selected features)
fetal$NSP <- factor(fetal$NSP)

#2 rows missing data on every variable: can remove
#last row missing some variables, doesn't have a classification label
fetal <- fetal[-c(2127,2128),]
dim(fetal) #2127 people, 20 features, 1 outcome variable
```

```
## [1] 2127   21
```

```
# drop the 1 observation with NA for NSP -> 2126 observations
fetal <- fetal |> drop_na(NSP)
```

```
# row is true class, column is predicted class

# all misclassifications are punished equally
loss_default <- matrix(c(0, 1, 1,
                         1, 0, 1,
                         1, 1, 0), nrow = 3, byrow = TRUE)

# we can change these values, like idk is a false negative 4x as bad as a false positive
loss_mild <- matrix(c(0, 1, 1,
                         3, 0, 1,
                         4, 2, 0), nrow = 3, byrow = TRUE)

loss_aggressive <- matrix(c(0, 1, 1,
                         12, 0, 1,
                         12, 3, 0), nrow = 3, byrow = TRUE)

calculate_loss <- function(true_labels, predicted_labels, loss_matrix) {
  true_labels <- as.numeric(as.character(true_labels))
  predicted_labels <- as.numeric(as.character(predicted_labels))
  mean(mapply(FUN = function(t, p) loss_matrix[t, p], true_labels, predicted_labels))
}
```

```
library(caret)
```

```
## Loading required package: lattice

##
## Attaching package: 'caret'

## The following object is masked from 'package:purrr':
##
##     lift
```

```
set.seed(263)

#80/20 split
train_indices <- createDataPartition(fetal$NSP, p = 0.8, list = FALSE)

fetal_train <- fetal[train_indices, ]
fetal_test  <- fetal[-train_indices, ]
```

Custom Asymmetric Loss Analysis

We have a classification problem with 3 classes of fetal state (1 = Normal, 2 = Suspect, 3 = Pathological). Classes 2 and 3 are underrepresented compared to class 1. Clinically, certain misclassifications are more harmful than others.

- Mistaking 3 as 1: Worst possible mistake
- Mistaking 2 as 1: Also a pretty bad mistake
- Mistaking 3 as 2: Bad, but not as bad as the previous two cases
- Mistaking 2 as 3: not that bad
- Mistaking 1 as 2: not that bad
- Mistaking 1 as 3: not that bad

Thus, we can define a custom loss that punishes certain mixups accordingly.

```
# row is true class, column is predicted class

# all misclassifications are punished equally
loss_default <- matrix(c(0, 1, 1,
                         1, 0, 1,
                         1, 1, 0), nrow = 3, byrow = TRUE)

# we can change these values, like idk is a false negative 4x as bad as a false positive
#4 times as bad
loss_mild <- matrix(c(0, 1, 1,
                         3, 0, 1,
                         4, 2, 0), nrow = 3, byrow = TRUE)

#12 times as bad
loss_aggressive <- matrix(c(0, 1, 1,
                         12, 0, 1,
                         12, 3, 0), nrow = 3, byrow = TRUE)

calculate_loss <- function(true_labels, predicted_labels, loss_matrix) {
  true_labels <- as.numeric(true_labels)
  predicted_labels <- as.numeric(predicted_labels)
  mean(mapply(FUN = function(t, p) loss_matrix[t, p], true_labels, predicted_labels))
}
```

CV to determine alternate probability thresholds

Okay so our best model with the best class accuracies is the aggressive loss model GBM model with n.trees=250, interaction.depth=4, shrinkage=0.2 (see fetal.code).

Can we make it any better by adjusting the probability thresholds?

We prioritize the class2 threshold since it is the hardest to get right, then class3, then class1.

```
library(gbm)
library(caret)
get_gbm_labels_threshold <- function(gbm_model, dataset, n.trees = 100, class2_threshold, class3_thresh
  probs <- predict.gbm(gbm_model, dataset, type = "response", n.trees = n.trees)[,,1]
  labels <- rep(1, nrow(probs))

  # adjust class 2 probability first
  labels[probs[, 2] >= class2_threshold] <- 2

  # then class 3, if wasn't already assigned to class 2
  labels[(probs[, 2] < class2_threshold) & probs[, 3] >= class3_threshold] <- 3
```

```r
  return(as.factor(labels))

}

set.seed(263)
# 5 folds
K <- 5
fold <- sample(1:K, nrow(fetal_train), replace = TRUE)

class2_thresholds <- c(0.1, 0.15, 0.2, 0.25, 0.3, 1/3)
class3_thresholds <- c(0.1, 0.15, 0.2, 0.25, 0.3, 1/3)

thresholds_results <- data.frame(threshold2 = c(), threshold3 = c(), loss_aggressive = c(), accuracy = c
                                 sensitivity2 = c(), sensitivity3 = c(), weighted_avg_sensitivity23 = c

for(p2 in class2_thresholds) {
  for (p3 in class3_thresholds) {
    fold_loss_aggressive <- c()
    fold_accuracy <- c()
    fold_sensitivity2 <- c()
    fold_sensitivity3 <- c()
     for(k in 1:K) {
        train <- fetal_train[fold != k, ]
        val <- fetal_train[fold == k, ]

        gbm_model <- gbm(NSP ~ ., data = train, distribution = "multinomial",
                         n.trees = 250, interaction.depth = 4, shrinkage = 0.2)
        val_predicted_label <- get_gbm_labels_threshold(gbm_model, val, n.trees = 250, p2, p3)
        fold_loss_aggressive[k] <- calculate_loss(val$NSP, val_predicted_label, loss_aggressive)
        fold_accuracy[k] <- mean(as.numeric(val_predicted_label) == as.numeric(val$NSP))
        fold_sensitivity2[k] <- mean(val_predicted_label[val$NSP == 2] == 2)
        fold_sensitivity3[k] <- mean(val_predicted_label[val$NSP == 3] == 3)

     }

    thresholds_results <- thresholds_results |> rbind(data.frame(
      threshold2 = p2,
      threshold3 = p3,
      loss_aggressive = mean(fold_loss_aggressive),
      accuracy = mean(fold_accuracy),
      sensitivity2 = mean(fold_sensitivity2),
      sensitivity3 = mean(fold_sensitivity3),
      # sensitivity of class 2 twice as important than class 3 idk
      weighted_avg_sensitivity23 = ((2/3) * mean(fold_sensitivity2) + (1/3) * mean(fold_sensitivity3)))
    )

  }
}

thresholds_results
```

```
##    threshold2 threshold3 loss_aggressive  accuracy sensitivity2 sensitivity3
## 1   0.1000000  0.1000000       0.3530347 0.9287241    0.8318796    0.9003367
```

```
## 2   0.1000000  0.1500000       0.3396242 0.9280573    0.8441702    0.9022637
## 3   0.1000000  0.2000000       0.3340403 0.9296923    0.8490604    0.9022637
## 4   0.1000000  0.2500000       0.3316072 0.9311084    0.8485891    0.8860510
## 5   0.1000000  0.3000000       0.3453618 0.9281353    0.8394083    0.8918470
## 6   0.1000000  0.3333333       0.3454121 0.9281822    0.8487429    0.8918470
## 7   0.1500000  0.1000000       0.3218416 0.9379648    0.8438272    0.9161857
## 8   0.1500000  0.1500000       0.3375240 0.9346759    0.8441446    0.8968735
## 9   0.1500000  0.2000000       0.3285349 0.9318473    0.8444523    0.9085137
## 10  0.1500000  0.2500000       0.3414786 0.9298852    0.8446808    0.9001473
## 11  0.1500000  0.3000000       0.3582676 0.9335178    0.8268015    0.8930044
## 12  0.1500000  0.3333333       0.3747166 0.9305457    0.8280955    0.8784542
## 13  0.2000000  0.1000000       0.3472745 0.9368698    0.8284426    0.9159211
## 14  0.2000000  0.1500000       0.3642407 0.9345075    0.8221102    0.8992544
## 15  0.2000000  0.2000000       0.3656187 0.9345201    0.8218760    0.8908129
## 16  0.2000000  0.2500000       0.3357486 0.9364534    0.8393475    0.9063973
## 17  0.2000000  0.3000000       0.3841855 0.9333774    0.8276511    0.8763378
## 18  0.2000000  0.3333333       0.3925766 0.9315595    0.8147354    0.8955748
## 19  0.2500000  0.1000000       0.3799930 0.9368564    0.8065986    0.9063973
## 20  0.2500000  0.1500000       0.3836320 0.9357726    0.8107256    0.8915825
## 21  0.2500000  0.2000000       0.4179571 0.9321641    0.8014275    0.8890602
## 22  0.2500000  0.2500000       0.3892652 0.9346224    0.8095290    0.8951209
## 23  0.2500000  0.3000000       0.4078238 0.9346158    0.8022459    0.9076209
## 24  0.2500000  0.3333333       0.8222054 0.9059489    0.6641873    0.7563973
## 25  0.3000000  0.1000000       0.3628544 0.9415790    0.8177477    0.9221711
## 26  0.3000000  0.1500000       0.3912831 0.9386619    0.8025324    0.9099357
## 27  0.3000000  0.2000000       0.4039344 0.9380121    0.8053654    0.9063973
## 28  0.3000000  0.2500000       0.3919096 0.9369773    0.8118050    0.8918470
## 29  0.3000000  0.3000000       0.4086499 0.9392722    0.8011467    0.8992544
## 30  0.3000000  0.3333333       0.8478184 0.9055247    0.6456829    0.7585137
## 31  0.3333333  0.1000000       0.4221416 0.9336581    0.7871616    0.8995190
## 32  0.3333333  0.1500000       0.4014133 0.9392515    0.8017450    0.9087783
## 33  0.3333333  0.2000000       0.3878903 0.9405055    0.8061273    0.9159211
## 34  0.3333333  0.2500000       0.3747540 0.9414926    0.8054007    0.9200547
## 35  0.3333333  0.3000000       0.4315365 0.9367983    0.7886303    0.8899952
## 36  0.3333333  0.3333333       0.4209864 0.9367659    0.7970479    0.8869438
##    weighted_avg_sensitivity23
## 1                   0.8546986
## 2                   0.8635347
## 3                   0.8667948
## 4                   0.8610764
## 5                   0.8568879
## 6                   0.8631110
## 7                   0.8679467
## 8                   0.8617209
## 9                   0.8658061
## 10                  0.8631696
## 11                  0.8488692
## 12                  0.8448818
## 13                  0.8576021
## 14                  0.8478250
## 15                  0.8448550
## 16                  0.8616974
## 17                  0.8438800
## 18                  0.8416818
```

```
## 19                  0.8398648
## 20                  0.8376779
## 21                  0.8306384
## 22                  0.8380596
## 23                  0.8373709
## 24                  0.6949239
## 25                  0.8525555
## 26                  0.8383335
## 27                  0.8390427
## 28                  0.8384856
## 29                  0.8338493
## 30                  0.6832931
## 31                  0.8246141
## 32                  0.8374227
## 33                  0.8427252
## 34                  0.8436187
## 35                  0.8224186
## 36                  0.8270132
```

Best model to maximize class 2 sensitivity

```
thresholds_results[which.max(thresholds_results$sensitivity2), ] # threshold2 = 0.1, threshold3 = 0.2 m
```

```
##   threshold2 threshold3 loss_aggressive  accuracy sensitivity2 sensitivity3
## 3        0.1        0.2       0.3340403 0.9296923    0.8490604    0.9022637
##   weighted_avg_sensitivity23
## 3                  0.8667948
```

```
set.seed(263)

best_gbm_aggressive_loss_model <- gbm(NSP ~ ., data = fetal_train, distribution = "multinomial",
                                      n.trees = 250, interaction.depth = 4, shrinkage = 0.2)
```

```
## Warning: Setting `distribution = "multinomial"` is ill-advised as it is
## currently broken. It exists only for backwards compatibility. Use at your own
## risk.
```

```
train_labels_sensitivity2 <- get_gbm_labels_threshold(best_gbm_aggressive_loss_model, fetal_train, 250,
test_labels_sensitivity2 <- get_gbm_labels_threshold(best_gbm_aggressive_loss_model, fetal_test, 250, 0

confusionMatrix(train_labels_sensitivity2, fetal_train$NSP) # 99.71 train acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##          1 1319    0    0
##          2    5  236    0
##          3    0    0  141
```

```
##
## Overall Statistics
##
##                Accuracy : 0.9971
##                  95% CI : (0.9932, 0.999)
##     No Information Rate : 0.7784
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9921
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9962   1.0000  1.00000
## Specificity            1.0000   0.9966  1.00000
## Pos Pred Value         1.0000   0.9793  1.00000
## Neg Pred Value         0.9869   1.0000  1.00000
## Prevalence             0.7784   0.1387  0.08289
## Detection Rate         0.7754   0.1387  0.08289
## Detection Prevalence   0.7754   0.1417  0.08289
## Balanced Accuracy      0.9981   0.9983  1.00000
```

```
# train class sensitivities: 0.9962   1.0000  1.00000
# train class balanced accuracies: 0.9981   0.9983  1.00000


confusionMatrix(test_labels_sensitivity2, fetal_test$NSP) # 92.71 test acc x
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2   3
##          1 310   4   2
##          2  20  55   4
##          3   1   0  29
##
## Overall Statistics
##
##                Accuracy : 0.9271
##                  95% CI : (0.8981, 0.9499)
##     No Information Rate : 0.7788
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.8126
##
##  Mcnemar's Test P-Value : 0.001817
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9366   0.9322  0.82857
## Specificity            0.9362   0.9344  0.99744
```

```
## Pos Pred Value         0.9810   0.6962  0.96667
## Neg Pred Value         0.8073   0.9884  0.98481
## Prevalence             0.7788   0.1388  0.08235
## Detection Rate         0.7294   0.1294  0.06824
## Detection Prevalence   0.7435   0.1859  0.07059
## Balanced Accuracy      0.9364   0.9333  0.91300
```

```
# test class sensitivities: 0.9366   0.9322  0.82857
# test class balanced accuracies: 0.9364   0.9333  0.91300

calculate_loss(fetal_train$NSP, train_labels_sensitivity2, loss_aggressive) # 0.002939447
```

```
## [1] 0.002939447
```

```
calculate_loss(fetal_test$NSP, test_labels_sensitivity2, loss_aggressive) # 0.2470588
```

```
## [1] 0.2470588
```

Best model to maximize avg of class 2 and class 3 sensitivity (weighted avg to prioritize 2), minimizes loss aggressive

These thresholds minimized loss aggressive while also maximizing the weighted average of the sensitivities of 2 and 3 (senstivity 2 was weighted twice that of 3, chosen arbitrarily idk)

```
thresholds_results[which.max(thresholds_results$weighted_avg_sensitivity23), ] # threshold2 = 0.15, thr
```

```
##   threshold2 threshold3 loss_aggressive  accuracy sensitivity2 sensitivity3
## 7       0.15        0.1       0.3218416 0.9379648    0.8438272    0.9161857
##   weighted_avg_sensitivity23
## 7                  0.8679467
```

```
set.seed(263)

train_labels_sensitivity23avg <- get_gbm_labels_threshold(best_gbm_aggressive_loss_model, fetal_train, 1
test_labels_sensitivity23avg <- get_gbm_labels_threshold(best_gbm_aggressive_loss_model, fetal_test, 250

confusionMatrix(train_labels_sensitivity23avg, fetal_train$NSP) # 99.88 train acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##          1 1322    0    0
##          2    2  236    0
##          3    0    0  141
##
## Overall Statistics
##
##                Accuracy : 0.9988
```

```
##                  95% CI : (0.9958, 0.9999)
##     No Information Rate : 0.7784
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9968
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9985   1.0000  1.00000
## Specificity            1.0000   0.9986  1.00000
## Pos Pred Value         1.0000   0.9916  1.00000
## Neg Pred Value         0.9947   1.0000  1.00000
## Prevalence             0.7784   0.1387  0.08289
## Detection Rate         0.7772   0.1387  0.08289
## Detection Prevalence   0.7772   0.1399  0.08289
## Balanced Accuracy      0.9992   0.9993  1.00000
```

```
# train class sensitivities: 0.9985   1.0000  1.00000
# train class balanced accuracies: 0.9992   0.9993  1.00000

confusionMatrix(test_labels_sensitivity23avg, fetal_test$NSP) # 93.41 test acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2   3
##          1 313   5   2
##          2  17  53   2
##          3   1   1  31
##
## Overall Statistics
##
##                Accuracy : 0.9341
##                  95% CI : (0.9062, 0.9558)
##     No Information Rate : 0.7788
##     P-Value [Acc > NIR] : < 2e-16
##
##                   Kappa : 0.8283
##
##  Mcnemar's Test P-Value : 0.06544
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9456   0.8983  0.88571
## Specificity            0.9255   0.9481  0.99487
## Pos Pred Value         0.9781   0.7361  0.93939
## Neg Pred Value         0.8286   0.9830  0.98980
## Prevalence             0.7788   0.1388  0.08235
## Detection Rate         0.7365   0.1247  0.07294
## Detection Prevalence   0.7529   0.1694  0.07765
```

```
## Balanced Accuracy      0.9356   0.9232  0.94029
```

```
# test class sensitivities: 0.9456   0.8983  0.88571
# test class balanced accuracies: 0.9356   0.9232  0.94029, nice lol

calculate_loss(fetal_train$NSP, train_labels_sensitivity23avg, loss_aggressive) #0.001175779
```

```
## [1] 0.001175779
```

```
calculate_loss(fetal_test$NSP, test_labels_sensitivity23avg, loss_aggressive) #0.2564706
```

```
## [1] 0.2564706
```

Default GBM probability thresholds

```
set.seed(263)

train_labels_sensitivity23avg <- get_gbm_labels_threshold(best_gbm_aggressive_loss_model, fetal_train, 1
test_labels_sensitivity23avg <- get_gbm_labels_threshold(best_gbm_aggressive_loss_model, fetal_test, 250

confusionMatrix(train_labels_sensitivity23avg, fetal_train$NSP) # train acc 99.88
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##          1 1322    0    0
##          2    2  236    0
##          3    0    0  141
##
## Overall Statistics
##
##                Accuracy : 0.9988
##                  95% CI : (0.9958, 0.9999)
##     No Information Rate : 0.7784
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9968
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9985   1.0000  1.00000
## Specificity            1.0000   0.9986  1.00000
## Pos Pred Value         1.0000   0.9916  1.00000
## Neg Pred Value         0.9947   1.0000  1.00000
## Prevalence             0.7784   0.1387  0.08289
```

```
## Detection Rate         0.7772   0.1387  0.08289
## Detection Prevalence   0.7772   0.1399  0.08289
## Balanced Accuracy      0.9992   0.9993  1.00000
```

```
# train class sensitivities: 0.9985   1.0000  1.00000
# train class balanced accuracies: 0.9992   0.9993  1.00000

confusionMatrix(test_labels_sensitivity23avg, fetal_test$NSP) # 94.35 test acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2   3
##          1 320   8   2
##          2  10  50   2
##          3   1   1  31
##
## Overall Statistics
##
##                Accuracy : 0.9435
##                  95% CI : (0.9171, 0.9635)
##     No Information Rate : 0.7788
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.8468
##
##  Mcnemar's Test P-Value : 0.8281
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9668   0.8475  0.88571
## Specificity            0.8936   0.9672  0.99487
## Pos Pred Value         0.9697   0.8065  0.93939
## Neg Pred Value         0.8842   0.9752  0.98980
## Prevalence             0.7788   0.1388  0.08235
## Detection Rate         0.7529   0.1176  0.07294
## Detection Prevalence   0.7765   0.1459  0.07765
## Balanced Accuracy      0.9302   0.9073  0.94029
```

```
# test class sensitivities: 0.9668   0.8475  0.88571
# test class balanced accuracies: 0.9302   0.9073  0.94029

calculate_loss(fetal_train$NSP, train_labels_sensitivity23avg, loss_aggressive) #0.0011757779
```

```
## [1] 0.001175779
```

```
calculate_loss(fetal_test$NSP, test_labels_sensitivity23avg, loss_aggressive) #0.3247059
```

```
## [1] 0.3247059
```

Conclusions: Using default probability thresholds on the best GBM aggressive model increases the sensitivity of class 2 while only slightlyyyy decreasing sensitivity1 (which is an okay tradeoff since 1 is normal), no

change to test sensitivity3. I think the best option is the threshold2 = 0.15, threshold3 = 0.1, which is the option that maximized the weighted average of sensivity2 and sensitivity3 (where senstivity2 was twice as important as sensitivity3, since 2 is the hardest class to get right), although this was arbitrarily chosen. This also minimized loss aggressive. Test sensitivies were (0.9668, 0.8475, 0.88571) using default thresholds and (0.9456, 0.8983, 0.88571) with threshold2 = 0.15, threshold3 = 0.1. This model also has very impressive balanced accuracies: (0.9356, 0.9232, 0.94029), which is an increase of (0.0054, 0.0159, 0) compared to the test balanced accuracies in the defaukkt threshold best aggressive GBM model.